Constructing a More Effective Online Bookstore Interface with Contextual SQL

Submitted for CS 3600 (Database Systems) at the University of Idaho

Ananda Shrestha
shre7751@vandals.uidaho.edu
University of Idaho
Moscow, Idaho, USA

Arjun Rai
rai1185@vandals.uidaho.edu
University of Idaho
Moscow, Idaho, USA

Jacob Kuenning
kuen5398@vandals.uidaho.edu
University of Idaho
Moscow, Idaho, USA

Abstract

E-commerce platforms often fail to take context into consideration when a customer of the platform searches for an item. That is, simple platforms making use of databases will only return exact matches, and not suggest alternatives (if there is no match) or similar items that might fulfill the customers needs. To address this limitation, we present the overview of a mock bookstore application that intelligently presents alternative books to users using principles of Type Abstraction Hierarchies (TAH) and numeric distance functions. Our proposed platform supplies users with a React front-end, and given search parameters, including title, author, page count, cost, and review score, the application processes these searches with a Python Flask backend before querying a MySQL database. By evaluating similarities in books using a genre TAH, and distance from searched for cost, page count, and publication date, the system will retrieve both exact matches and similar books corresponding to the users search.

1 Introduction

In the modern online retail environment, it is important to both customers and retailers that searches of their databases through their online platforms are flexible and take into consideration some context. Customers are searching retailer databases with multidimensional criteria, and are expecting relevant results even when there are no exact matches. However, the limitations of SQL make some interpretation from search terms to SQL query necessary. Queries of relational databases, like those made with SQL, can only return either an empty set or exact matches. SQL alone does not do a sufficient job holding context or evaluating the proximity of two entities.

Without context, the experience of both the retailer and the customer is diminished. The customer expects alternatives to be suggested to them and might become frustrated when faced with an empty page. The retailer misses the opportunity to provide customers with alternatives that they might wish to purchase, missing out on potential revenue. To avoid this, such applications must also intelligently return suggested, related items.

To address the limitations of basic SQL, we propose a plan for a mock bookstore application that uses context-aware search features. Rather than using strict SQL matching, our project will take the search parameters from the users and process them, using Python Flask, taking advantage of relationships between objects, including distance from numeric values (example: cost, page count) and similarity based on a TAH (genre). The resulting SQL query will return exact matches and similar books, ranked by their similarity.

On the user side, our mock customers will be able to search our database using parameters such as title, author, cost, publication date, genre, page count, and review score. They can choose which parameters they can relax, meaning which attributes they are okay with not getting a perfect match on.

This Phase I report details the ER Diagram that we will be constructing our database from, our project plan and timeline, and our specific software stack.

2 Background

SmartBase is built upon Dr. Hasan Jamil's Contextual SQL (ConSQL) framework to provide cooperative, flexible searches. The system relies on two main theoretical pillars:

2.1 Type Abstraction Hierarchies (TAH)

A tree-based categorization for categorical attributes (e.g., *Epic Fantasy* → *Fantasy* → *Fiction*). This allows the query engine to dynamically substitute missing subgenres with broader parent categories

2.2 Numeric Distance Functions (δ^t)

Mathematical formulas that measure numerical closeness (such as MSRP and review scores) on a normalized 0.0 to 1.0 scale.

3 ER Diagram

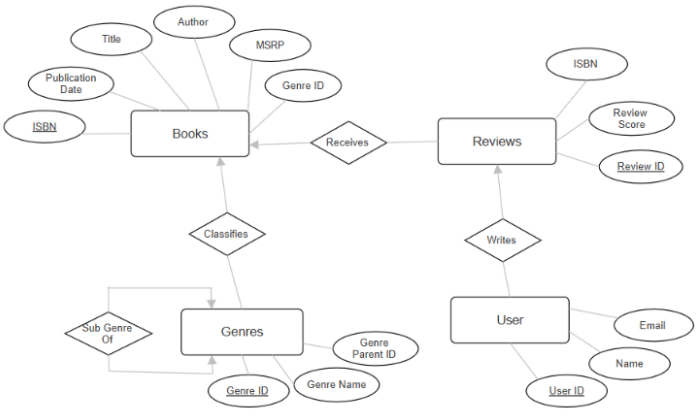


Figure 1: Entity-Relationship diagram for the SmartBase bookstore application.

This is our ER Model for our SmartBase bookstore project for the databases class at the University of Idaho. This model was designed

for a relational database using context-aware search features, which are used when the focus is on handling multi-criteria searches and query relaxation through Type Abstraction Hierarchies (TAH) and numeric distance functions.

3.1 Entities

Our entities that we will be storing are Books, Genres, Reviews, and User. For Books, they will have the properties ISBN, Title, Publication Date, Author, MSRP, and Genre ID, with ISBN being the primary key. Genres will have the properties Genre ID, Genre Name, and Genre Parent ID, with Genre ID being the primary key. Reviews will have the properties Review ID, Review Score, ISBN, and User ID, with Review ID being the primary key. For User, it will have the properties User ID, Name, and Email, with User ID being the primary key.

3.2 Relations

Our relations that we will be storing are Classifies, Sub Genre Of, Receives, and Writes. The Classifies relation maps Genres to Books, while Sub Genre Of establishes a recursive parent-child hierarchy within the Genres table for query relaxation. The Receives relation connects Reviews to Books, and Writes connects User to Reviews to track user feedback.

4 Project Plan and Timeline

4.1 October 31: Phase II

By this date, we will have a working prototype of our application. We will have the tables populated in our MySQL database. We will also have a simple front-end user interface where users can perform searches. We will have the numeric distance functions and TAH hierarchy implemented in Python. The pipeline from search to Flask to MySQL and back will be finished.

4.2 November 21: Phase III

By this date, we will have completed fixing the bugs and errors present in our prototype. We will also implement the user review feature, where users of the program can log in and leave reviews.

4.3 December 10: Phase IV

By this date, we will have the project complete. The UI will be redone to make it user-friendly and clean, and we will present our finished project to the class.

5 Software Stack

5.1 Front End

For our front end, we will be using the JavaScript framework React. We chose this as our front-end technology because it is widely adopted by the industry and features a wide ecosystem of libraries and packages. In addition, React allows us to update search results dynamically on the page without requiring a full browser refresh.

5.2 Back End

For our back end, we chose Python Flask. We chose this for easy integration with React and because using Python for our back end will allow us to quickly prototype and iterate. Python also provides a straightforward environment to execute our custom Type Abstraction Hierarchy (TAH) and numeric distance algorithms before sending queries to the database.

5.3 Database

For our database, we chose MySQL. We chose MySQL for its widespread adoption and easy-to-connect tooling. Since MySQL is an industry standard, it allows us to directly demonstrate how custom middleware can add contextual search capabilities to traditional relational databases.

6 Conclusion

In this paper, we outlined our project for creating a context-aware application for users to search an online bookseller's database. We described why such an application is important to both customers and online retailers, and how traditional database searching applications fall short of expectations. We also provided background on the technologies needed to make this a reality.

Described in this paper is our ER diagram and the purposes of its entities and relations. Also described is our software stack and the justifications for using these specific technologies. Finally, we provided our project plan and development timeline.

References

Hasan M. Jamil, Context as a Distance Function in ConSQL. ISMIS 2022: 240-248" (DOI: 10.1007/978-3-031-16564-1_23)